

Web Development

Self-Assessment Exam

Python · Django · REST · Databases · React · Vite · Deployment

Based on the Green Hills Training System codebase

Prepared for Yassin

Part A — Questions (answer first, no peeking)

Part B — Full Answer Key (check yourself after)

How to take this exam

This exam has 60 questions across 9 sections, mirroring the structure of the study course. Take it the way an interviewer would push you: answer out loud (or in writing) in your own words, then check Part B for the canonical answer.

Recommended workflow:

1. Write your answers on paper or in a Word doc *before* looking at Part B.
2. Score yourself honestly. Anything where you wrote “no idea” gets a 0.
3. Re-read the corresponding chapter of the study course for every question you missed.
4. Take the exam again a week later. Track your score.

Scoring guide:

48+ correct — ready for a junior dev interview.

36–47 — solid foundation, polish the weak spots.

24–35 — keep studying, retake in a week.

Below 24 — re-read the course, no rush. You’ll get there.

PART A — Exam Questions

Answer each question in your own words. Do not look at Part B until you have finished all 60.

Section 1 — How the web works (1–6)

1. Define the client-server model in one sentence.
2. List the four HTTP methods you use most often and one sentence about what each does.
3. What does each of these status codes mean: 200, 201, 400, 401, 403, 404, 500?
4. What is JSON, and why is it the dominant format for web APIs?
5. In one sentence, what is REST?
6. If a request includes a body, which HTTP method is NOT typically used to send one: GET, POST, PUT, PATCH, DELETE? Why?

Section 2 — Python essentials (7–13)

7. Which of these are mutable, which are immutable? — list, tuple, dict, string, int.
8. What does **self** refer to in a Python class?
9. What is the purpose of `__init__` in a class?
10. Explain a Python decorator in one sentence. Give one example you have seen in Django.
11. Why use a virtual environment? Name one consequence of not using one.
12. What does `pip install -r requirements.txt` do, and why is the file named that way?
13. Look at this snippet. What gets printed?

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

Section 3 — Django (14–30)

14. Define **project** and **app** in Django. Give an example of each from the training system.
15. Where is the list of registered apps located, and what is that setting called?
16. Name 4 commands you regularly run with **manage.py** and what each does.
17. What is the Django ORM in one sentence?
18. What is the difference between **makemigrations** and **migrate**?
19. Write the ORM query for “all competencies that require an exam, ordered by priority descending”.

20. Explain what **select_related** does and what problem it prevents.
21. In **on_delete**, what is the difference between CASCADE, SET_NULL, and PROTECT?
22. Function-based view vs class-based view (APIView): when do you pick which?
23. ViewSet (with a router) vs APIView: when do you pick which?
24. What are the TWO jobs a DRF serializer performs?
25. Where should validation logic live: in the view, the serializer, or the model? Why?
26. Where should authorization checks live: backend, frontend, or both? Why?
27. Explain how JWT authentication works in your project, from login to making a later API call.
28. What is the difference between **STATIC** files and **MEDIA** files?
29. Where does the routing of **GET /api/training/competencies/** happen? Name the two files involved.
30. What does **include()** do inside a `urls.py`?

Section 4 — Database (31–36)

31. Define **primary key** and **foreign key** in your own words.
32. Give an example of a one-to-many relationship in the training system, and an example of a many-to-many.
33. Write the SQL to fetch the username and position name for every user, using a join.
34. What is the practical difference between SQLite and Postgres? Why does your project use one in dev and the other in prod?
35. What is the N+1 query problem? Give the symptom and the fix in Django.
36. What is a database transaction, and when do you absolutely need one?

Section 5 — React (37–45)

37. In one sentence, what is React?
38. Is JSX HTML? What does the browser actually run?
39. What is the difference between **props** and **state**?
40. What does **useState** return, and why is it returned as a pair?
41. What does **useEffect** do, and what does an empty dependency array (`[]`) mean?
42. Why does React require a **key** prop on items rendered from an array? What happens without it?
43. What does **e.preventDefault()** do inside a form's submit handler?

- 44. How does your project attach the JWT token to every outgoing axios request without each page having to do it manually?
- 45. What is single-page routing, and which library handles it in your project?

Section 6 — Vite and the build (46–48)

- 46. What does **vite build** produce, and where does the output go?
- 47. What is the difference between **npm install** and **npm ci**?
- 48. When a user visits a deep link like **/manager/exams/5/questions** in production, who answers? Walk through the steps.

Section 7 — The training system (49–53)

- 49. Give your 30-second pitch of the training system.
- 50. What lives in **core/**, and how is it different from **accounts/**, **branches/**, **training/**?
- 51. List 5 important models in **training/models.py** and one sentence about each.
- 52. Walk through the flow when a manager grades an exam, from clicking Grade to the database being updated.
- 53. Why did your Employee Lookup tab show “No competencies assigned” even though the position had competencies? Where was the bug?

Section 8 — Deployment (54–57)

- 54. Walk through the deploy pipeline on Render, from **git push** to traffic switching.
- 55. Why are secrets stored as environment variables and not in **settings.py**?
- 56. How are user-uploaded files (profile pictures, PDFs) stored in production, and why not on Render’s disk?
- 57. What is the role of **WhiteNoise** in production?

Section 9 — Interview scenarios (58–60)

- 58. Interviewer asks: “How would you add a new feature where managers can leave notes on an employee’s profile?” Outline your approach (backend + frontend) in 6–8 lines.
- 59. Interviewer shows you this code and asks what it does and what could go wrong. Answer concisely:

```
users = User.objects.all()
for u in users:
    print(u.position.name)
```

- 60. Interviewer asks: “You have never used Celery. We use it for background jobs. How would you approach learning it on the job?”

PART B — Full Answer Key

Score yourself honestly. If your answer captures the same idea even with different wording, count it as correct.

Section 1 — Web fundamentals

1. Two computers talk over HTTP: a **client** (usually a browser) asks for things, a **server** (your Django app) responds. That request-response loop is the entire web.
2. **GET** = read; **POST** = create; **PUT** = replace entirely; **PATCH** = modify partially; **DELETE** = remove.
3. **200** OK; **201** Created; **400** Bad Request (client sent garbage); **401** not logged in; **403** logged in but not allowed; **404** not found; **500** server error (your code crashed).
4. JSON is a text format for structured data that maps cleanly to both Python dicts and JavaScript objects. It is dominant because both ends of a web call can read and write it natively.
5. A convention for designing HTTP APIs where URLs identify resources and HTTP methods describe the action on them.
6. **GET** — GET requests are for reading and have no body. Anything that needs a body uses POST/PUT/PATCH.

Section 2 — Python essentials

7. Mutable: **list**, **dict**. Immutable: **tuple**, **string**, **int**.
8. The instance the method is being called on. It is the first parameter of every instance method.
9. It is the constructor. Python calls it automatically when you create an object, e.g. **User("Yassin", "ADMIN")**.
10. A function that wraps another function to add behavior. Example: **@login_required** on a Django view, or **@api_view(["GET"])** in DRF.
11. It isolates your project's dependencies. Without it, two projects needing different versions of Django would conflict at the system level.
12. It installs every package listed in **requirements.txt** at the versions specified. The standard filename means anyone who clones the repo can recreate the same environment with one command.
13. **[1, 2, 3, 4]**. **y = x** does not copy the list; both names refer to the same list object, so appending through **y** shows up in **x**.

Section 3 — Django

14. **Project** = the whole site (one per repo); **app** = a self-contained feature (many per project). In your code, **core** is the project; **accounts**, **branches**, **training** are apps.
15. **INSTALLED_APPS**, in **core/settings.py**.
16. **runserver** (start dev server), **startapp X** (scaffold a new app), **makemigrations** + **migrate** (database changes), **createsuperuser** (make an admin), **collectstatic** (gather static files), **shell** (interactive Python with Django loaded).
17. A Python interface to the database: model classes become tables, queries are written in Python, and Django translates them to SQL.
18. **makemigrations** writes a migration file describing model changes; **migrate** applies those files to the database. The split exists so migration files can be committed to git and applied by teammates.
19. **Competency.objects.filter(requires_exam=True).order_by("-priority_points")**
20. It tells Django to fetch related foreign-key objects in the same SQL query using a JOIN, preventing the N+1 query problem.
21. **CASCADE**: delete children too. **SET_NULL**: set the FK to NULL on parent delete (needs **null=True**). **PROTECT**: refuse to delete the parent if children exist.
22. FBV is concise for tiny endpoints. CBV (APIView) is cleaner when an endpoint serves multiple HTTP methods, needs permission classes, or shares logic with other views. In your project, every endpoint is a CBV.
23. Pick a **ViewSet** when you want standard CRUD for a model (list/retrieve/create/update/delete — the router gives them all to you). Pick an **APIView** for a single custom action that doesn't fit CRUD (login, refresh, grade, promote).
24. (1) Take a Python object and turn it into JSON. (2) Take incoming JSON, validate it, and turn it into a Python object.
25. Usually in the **serializer** — that is DRF's job, and it produces clean error messages. Use the **model** for fundamental data integrity (e.g. **unique=True**) so it is enforced even from Django admin or other entry points.
26. Always on the **backend**. Anyone can call your API from DevTools or Postman, so frontend checks are UX only. Hide buttons in React for UX; enforce **permission_classes** in the view for security.
27. Login POST returns **{access, refresh}**. Frontend stores both in localStorage. axios has a request interceptor that reads the access token from localStorage and adds **Authorization: Bearer <token>** to every outgoing request. Backend decodes the token, identifies the user. When the access token expires, frontend uses the refresh token to get a new one without re-prompting for the password.

- 28. **STATIC** files come from you (CSS, JS, the React bundle). **MEDIA** files come from users (uploads). In your project, static is served by WhiteNoise, media by Cloudflare R2.
- 29. **core/urls.py** matches the prefix **api/training/** and delegates the rest of the path; **training/urls.py** matches **competencies/** and routes to the view in **training/views.py**.
- 30. It mounts the URL patterns of another module (usually an app's `urls.py`) under a path prefix. Lets each app own its own routing.

Section 4 — Database

- 31. **Primary key**: a column whose value uniquely identifies a row (Django adds an auto-increment **id** automatically). **Foreign key**: a column whose value is the primary key of a row in another table — how tables are connected.
- 32. One-to-many: **User** belongs to one **Branch**; one Branch has many Users. Many-to-many: **ExamTemplate** can be available in many **Branches**; a Branch can offer many ExamTemplates.
- 33. **SELECT u.username, p.name FROM users u JOIN positions p ON u.position_id = p.id;**
- 34. SQLite is a single file with no server — perfect for development and tiny projects. Postgres is a real database server that handles concurrent users, large data, complex queries. Production uses Postgres because the app will have multiple users hitting it at once.
- 35. When you loop over **N** rows and each iteration triggers an extra query (e.g. **row.foreign_key.field**), you end up running **1 + N** queries. The fix in Django: use **select_related()** for one-to-one/foreign-key relations or **prefetch_related()** for many-to-many.
- 36. A group of database operations that either all succeed or all fail together. Use one when two writes must stay in sync, e.g. creating a user AND giving them a starting wallet. Wrap the block in **with transaction.atomic():**.

Section 5 — React

- 37. A JavaScript library for building user interfaces from reusable components, where the UI is described as a function of state.
- 38. No. JSX is a syntax that compiles to **React.createElement(...)** calls. The browser runs the compiled JavaScript, not JSX.
- 39. **Props** are inputs passed in from a parent; they are read-only inside the component. **State** is data owned by the component itself; updating it triggers a re-render.
- 40. An array of two items: the current value and a setter function. It is a pair so you can give each piece of state its own name with destructuring (**const [count, setCount] = useState(0)**).
- 41. **useEffect** runs side effects (fetching data, subscribing, timers) after a render. An empty dependency array means “run once when the component mounts, never again.”

- 42. **key** lets React identify which items changed/moved between renders. Without it, React falls back to index-based matching and may re-use the wrong DOM nodes, causing flicker, wrong state, or input fields losing focus.
- 43. It stops the browser's default behavior, which for a form submit is a full page reload. Without it, your fetch/axios call never gets a chance to run before the page navigates away.
- 44. **frontend/src/api/client.js** defines an axios instance with a **request interceptor** that reads the token from **localStorage** and adds an **Authorization: Bearer <token>** header before each request goes out.
- 45. Single-page routing means the browser never does a full page reload between routes; JavaScript swaps which component renders for the current URL. **react-router-dom** handles it in your project.

Section 6 — Vite and the build

- 46. It produces a hashed, minified, production-ready bundle in **frontend/dist/** — HTML, JS, CSS, plus assets copied from **public/**.
- 47. **npm install** reads **package.json** and can update **package-lock.json**. **npm ci** reads **package-lock.json** and installs the exact versions, faster and deterministic. CI/CD (like your **build.sh**) uses **npm ci**.
- 48. Render's unicorn process receives the request. The URL is not **/api/**, **/admin/**, **/static/**, or **/media/**, so Django's SPA catch-all matches it and returns **index.html**. The browser parses **index.html**, loads the React bundle, React Router sees **/manager/exams/5/questions** and renders the corresponding component.

Section 7 — The training system

- 49. "Green Hills Training is a full-stack web app where each employee at a branch holds a position, and each position has required competencies. Employees train, take exams, and earn points that determine their level. Managers publish requirements, look up employees, and grade exams. Backend: Django + DRF. Frontend: React + Vite. Database: SQLite locally, Postgres in production. Media: Cloudflare R2. Deployed on Render with PWA support."
- 50. **core/** is the project — settings, master URL routing, WSGI/ASGI entry points. The others are *apps* — self-contained features that own their own models, views, serializers, and urls.
- 51. **Competency** (the thing to learn). **PositionCompetencyRequirement** (links position+branch to required competencies). **EmployeeCompetencyRequirement** (per-employee overrides). **EmployeeCompetencyRecord** (proof of passing). **ExamTemplate/Question/Choice/Answer/ExamSession** (exam content and attempts).
- 52. Manager opens Grading Queue (table of SUBMITTED sessions). Clicks Grade → modal fetches all answers for the session. For each answer, manager enters points + comment, Save POSTs to grade endpoint. Finalize calls grade-session endpoint — the session moves to GRADED, final score is computed, and if the exam is linked to a competency and the score ≥

60%, an **EmployeeCompetencyRecord** is created/updated.

53. Backend bug. The frontend was reading **lookupResult.competencies**, but **EmployeeActivityView** only returned **missing_competencies**. Fix: have the view merge **PositionCompetencyRequirement** + **EmployeeCompetencyRequirement** and return them as **competencies**, matching the employee-side endpoint.

Section 8 — Deployment

54. You push to GitHub. Render detects the push, pulls the new code, runs **build.sh** in a fresh container (pip install → npm ci & build frontend → collectstatic → migrate). Render starts gunicorn. Once health checks pass, Render switches traffic to the new version.
55. Security: anything in **settings.py** goes into git history forever, including secrets. Deployment flexibility: env vars let dev/staging/prod use the same code with different values (SQLite locally, Postgres in prod).
56. Render's free tier disk is ephemeral — files disappear on every deploy. Files are routed to **Cloudflare R2** (S3-compatible object storage) via **django-storages**. The database stores only the path; the bytes live in R2.
57. WhiteNoise lets Django serve its own static files (your CSS, JS, the built React bundle) efficiently in production, without needing a separate nginx or CDN layer.

Section 9 — Interview scenarios

58. "Backend: add a **Note** model with **employee** FK to User, **author** FK to User, **text** TextField, **created_at**. Run makemigrations + migrate. Add a serializer, an APIView or ViewSet at **/api/training/notes/**, and a permission class that restricts to managers and admins. Frontend: add a Notes card on the Employee Lookup tab in **ManagerDashboard.jsx**. Fetch existing notes on lookup, render them in a list, add a textarea + Save button that POSTs to the new endpoint, then refresh the list. Deploy and test."
59. It loops over every user and prints their position name. The problem: classic **N+1**. Accessing **u.position.name** on each iteration triggers an extra query, so a thousand users mean ~1001 queries. Fix: **User.objects.select_related("position")**. Bonus issue: if any user has **position=None**, **u.position.name** raises **AttributeError** — handle that case (e.g. **u.position.name if u.position else ""**).
60. "I'd start by reading the official Celery quickstart and the project's existing Celery usage to learn its conventions. Then I'd shadow a simple task end-to-end — how it gets queued, where the worker runs, how results are stored. I'd ask the senior responsible for it to walk me through one production task, then write a small toy task myself before touching anything real. Most importantly, I would say so — I'd rather ask the team than guess and break something."

Closing

If you scored under target, do not be discouraged — you took the exam. Most candidates never test themselves; they show up to the interview blind. Re-read the chapters that tripped you up, take the exam again in a week, and watch your score climb.

When you're comfortable with this material, ask me to design a harder exam (code-reading + debugging questions) or a mock interview where I play the senior dev and you walk me through your project.